

WINDOWS MANAGED WITH ANSIBLE

Project Description

This project consists of managing Windows machines from an Ubuntu server using Ansible. The connection is done through WinRM over HTTPS and uses Kerberos authentication inside the domain. The goal is to build a solution that is not only functional, also automated.

Objective and Advantages

The main objective is to avoid managing each machine manually and instead control everything from one place. With this setup, Windows machines can be prepared and configured automatically using GPO and certificates.

This solution has several advantages. It is secure, because all communication is done over HTTPS (encrypted) and uses Kerberos for authentication, avoiding insecure methods. It is also scalable, since the same configuration can be applied to many machines without repeating steps. In addition, it allows centralized management, where all actions are executed from a single Ubuntu server using Ansible.

Overall, it provides a clean, automated and realistic way to manage Windows systems in a domain environment.

Manual one-time tasks

- Install and configure AD CS
- Create OU
- Create and link GPO
- Duplicate and publish certificate template

Automated tasks

- Client receives GPO
- Auto-enrolls certificate
- WinRM HTTPS configuration
- Firewall rule application
- Validation through Ansible

Index:

1. Ubuntu Control Node Preparation	3
2. Kerberos Configuration	4
3. Windows Domain Preparation	5
4. Certificate Services Configuration	8
5. GPO and Autoenrollment	9
6. Firewall Configuration	12
7. Trusting the CA in Ubuntu	14
8. Ansible Script	15
9. Maintenance	16

1. Ubuntu Control Node Preparation

Steps to install Ansible on Ubuntu as a control node:

sudo apt update

```
hugo@hugo-server:~$ sudo apt update
```

sudo apt install software-properties-common -y

What it does is set up Ubuntu so you can easily add external repositories

```
hugo@hugo-server:~$ sudo apt install software-properties-common -y
```

sudo add-apt-repository --yes --update ppa:ansible/ansible

Add the repository from which you can download Ansible to Ubuntu and update the package list

```
hugo@hugo-server:~$ sudo add-apt-repository --yes --update ppa:ansible/ansible
```

sudo apt install ansible -y

```
hugo@hugo-server:~$ sudo apt install ansible -y
```

ansible --version

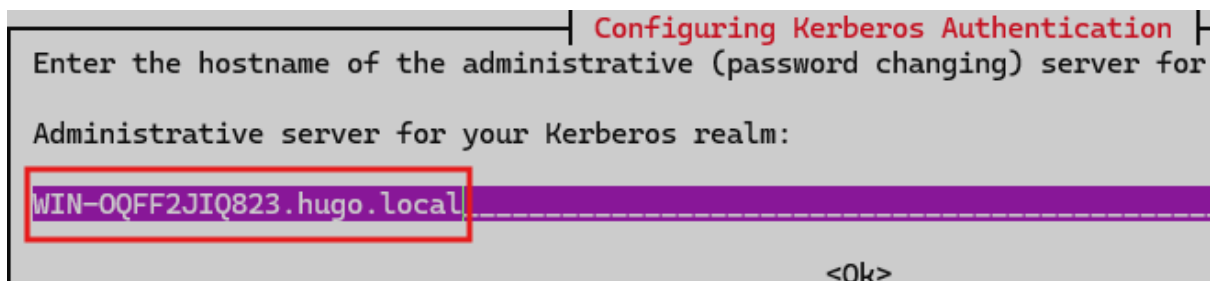
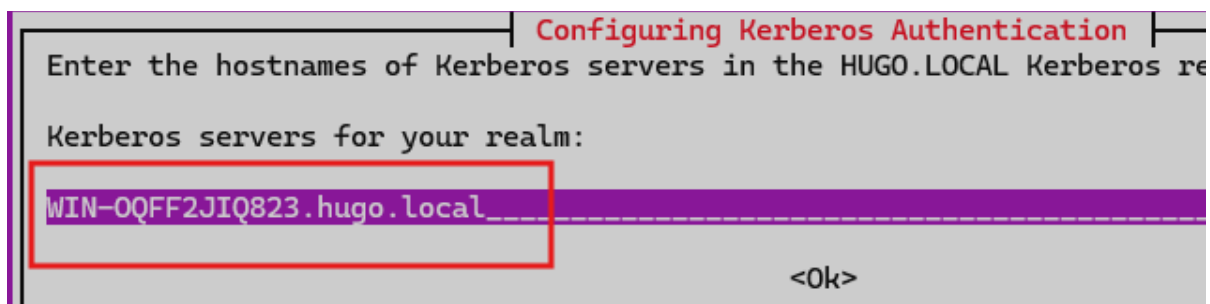
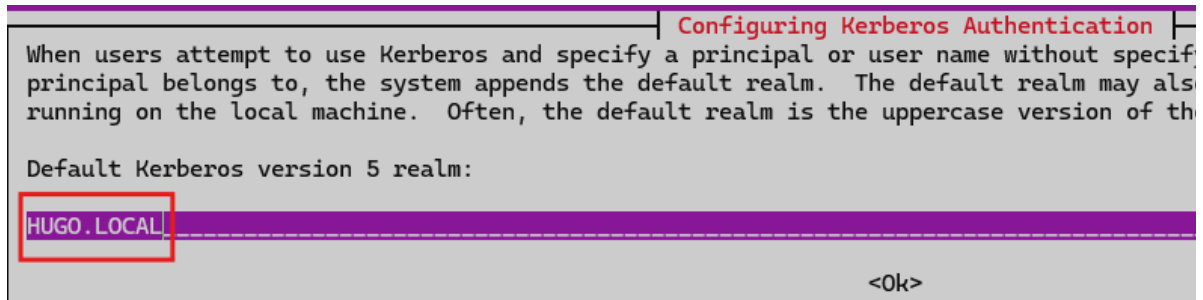
To check if it has been installed correctly and view the version

```
hugo@hugo-server:~$ ansible --version
ansible [core 2.20.3]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/hugo/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/hugo/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  pyyaml version = 6.0.1 (with libyaml v0.2.5)
hugo@hugo-server:~$
```

2. Kerberos Configuration

Now you need to install the necessary packages for Kerberos and WinRM.

sudo apt install krb5-user python3-pip -y



Try authenticating Ubuntu against the domain using Kerberos

kinit Administrator@HUGO.LOCAL

```
hugo@hugo-server:~$ kinit Administrator@HUGO.LOCAL
Password for Administrator@HUGO.LOCAL:
hugo@hugo-server:~$
```

klist

Shows the Kerberos ticket that Ubuntu just obtained

```

hugo@hugo-server:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: Administrator@HUGO.LOCAL

Valid starting    Expires          Service principal
03/16/2026 12:29:20  03/16/2026 22:29:20  krbtgt/HUGO.LOCAL@HUGO.LOCAL
        renew until 03/17/2026 12:29:15
hugo@hugo-server:~$ |

```

To manage the machines, you need to add them to the [inventory.ini](#)

```

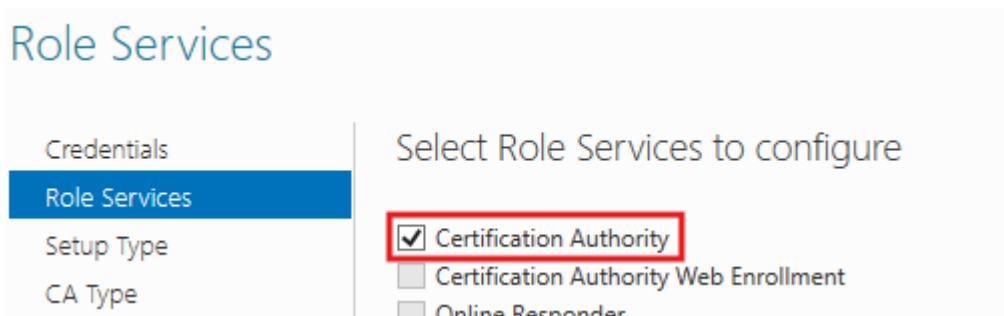
GNU nano 7.2
[windows]
winserver ansible_host=WIN-QQFF2JIQ823.hugo.local
wincliente ansible_host=WIN-ANSIBLE-01.hugo.local

[windows:vars]
ansible_connection=psrp
ansible_port=5986
ansible_psrp_auth=kerberos
ansible_user=Administrator@HUGO.LOCAL

```

3. Windows Domain Preparation

The first step is to install a valid server certificate on Windows. Without it, you cannot run WinRM over HTTPS; Microsoft requires a Server Authentication certificate whose CN matches the computer's hostname [DOCUMENTATION](#)



To configure the following roles, role services, or features, click Configure.

⤴ Active Directory Certificate Services

Certification Authority

CA Type:	Enterprise Root
Cryptographic provider:	RSA#Microsoft Software Key Storage Provider
Hash Algorithm:	SHA256
Key Length:	2048
Allow Administrator Interaction:	Disabled
Certificate Validity Period:	16/03/2031 6:24:00
Distinguished Name:	CN=hugo-WIN-OQFF2JIQ823-CA,DC=hugo,DC=local
Certificate Database Location:	C:\Windows\system32\CertLog
Certificate Database Log Location:	C:\Windows\system32\CertLog

The following roles, role services, or features were configured:

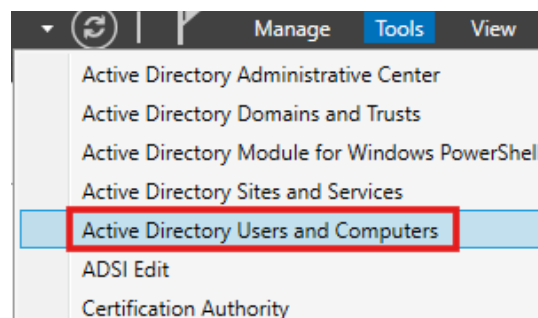
⤴ Active Directory Certificate Services

Certification Authority

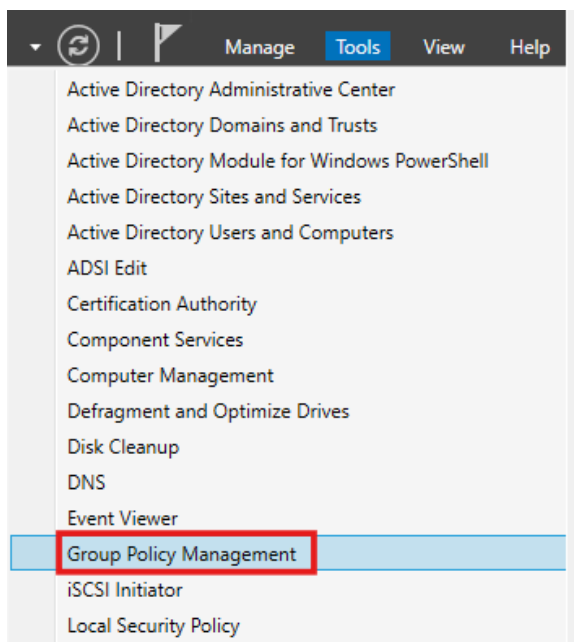
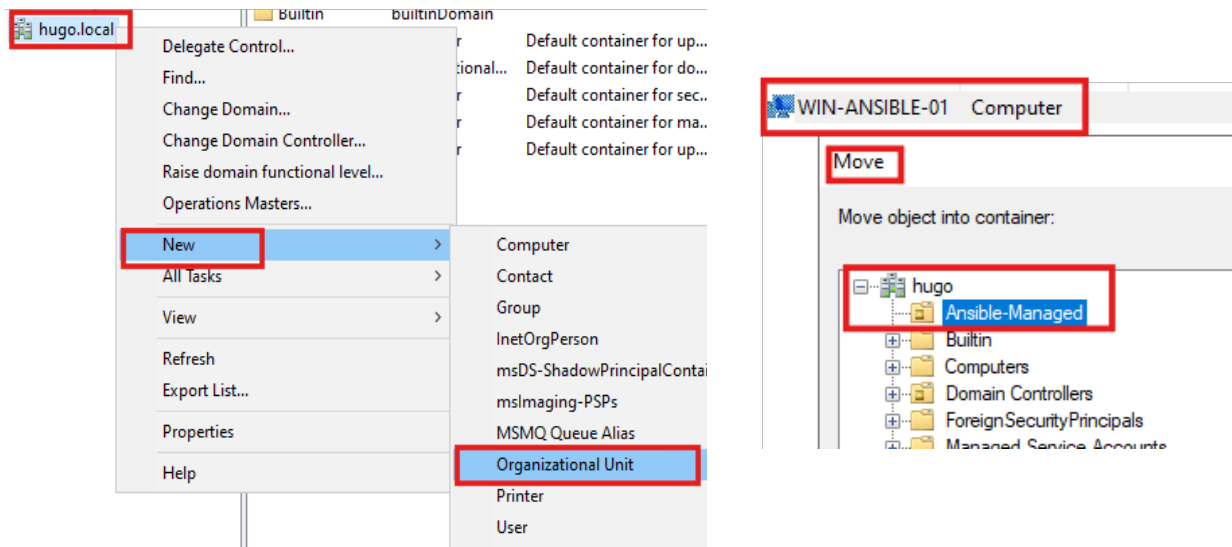
[More about CA Configuration](#)

✔ Configuration succeeded

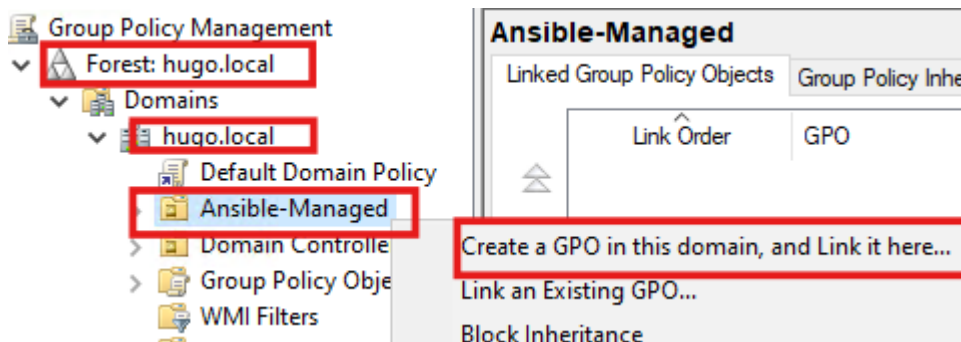
Now you need to **create an OU** specifically for the machines that will be managed via Ansible.

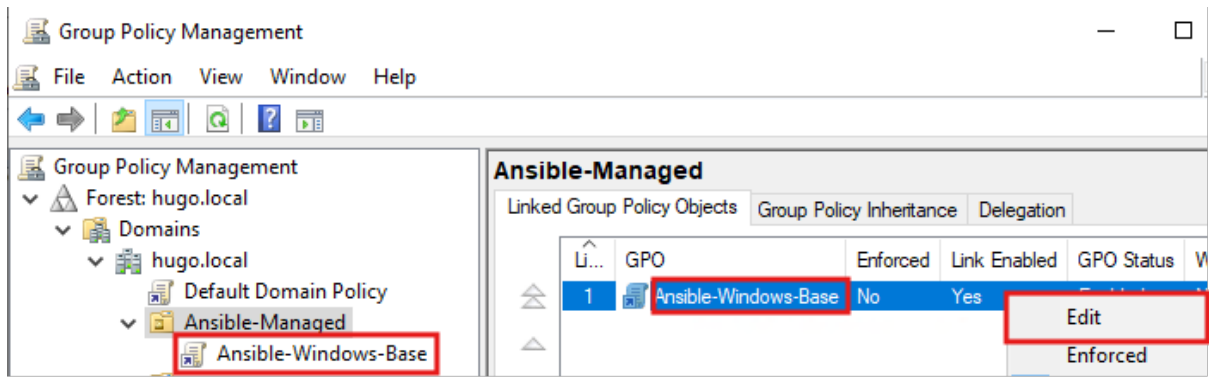


We create it within the domain. Then I add the computer to the OU we just created.



Now we need to create a **GPO** that will be linked to that OU. The idea is for that policy to automatically prepare the machines in that group for Ansible.

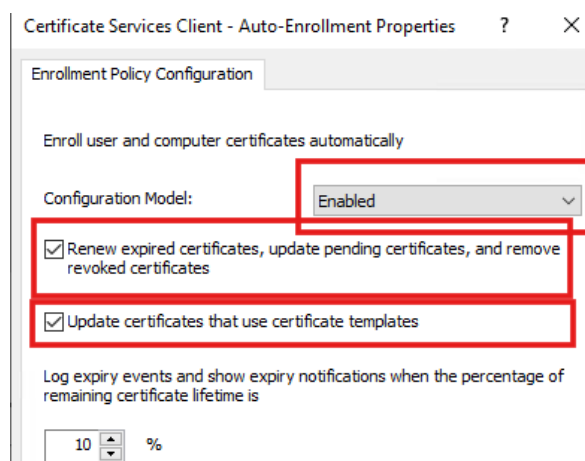
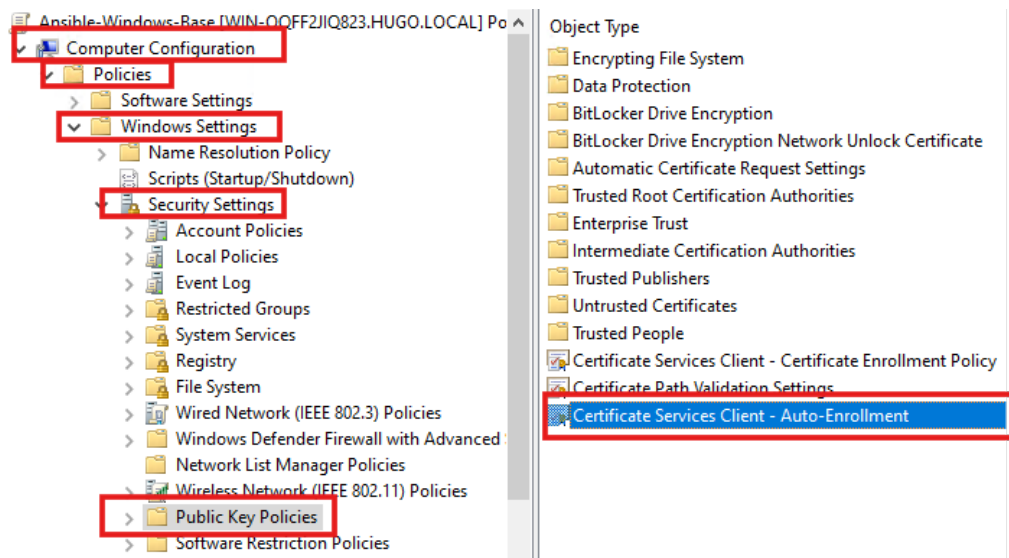




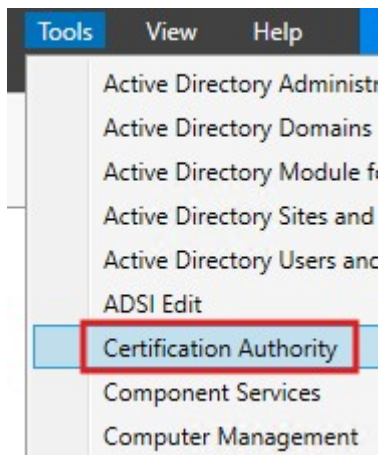
4. Certificate Services Configuration

For a Windows client to use WinRM over HTTPS, it must have a valid server certificate issued by the domain's CA.

Políticas → Windows Settings → Security Settings → Public Key Policies → Certificate Services Client - Certificate Enrollment Policy

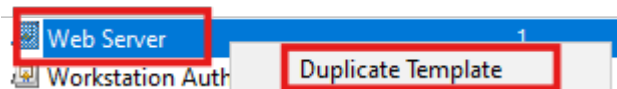


5. GPO and Autoenrollment

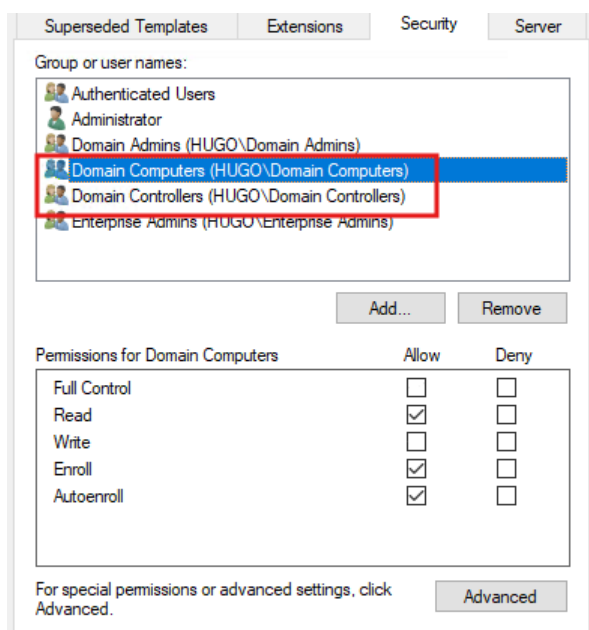
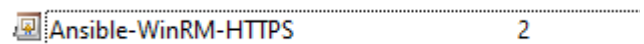


Now you need to set up the **server certificate that WinRM will use over HTTPS**.

We need to duplicate the WebServer template:



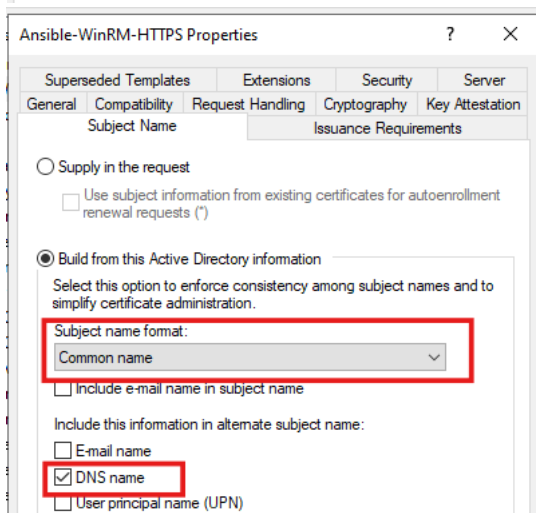
And I change the name and security settings:



Enroll allows the device to **request** that **certificate**.

Autoenroll allows the device to **request and renew** it automatically without manual intervention.

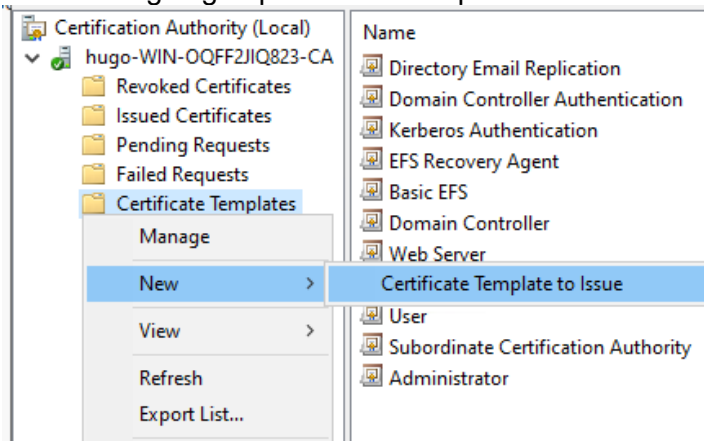
We have to put these permissions in both, in **Domain computers** and **Domain controllers**.



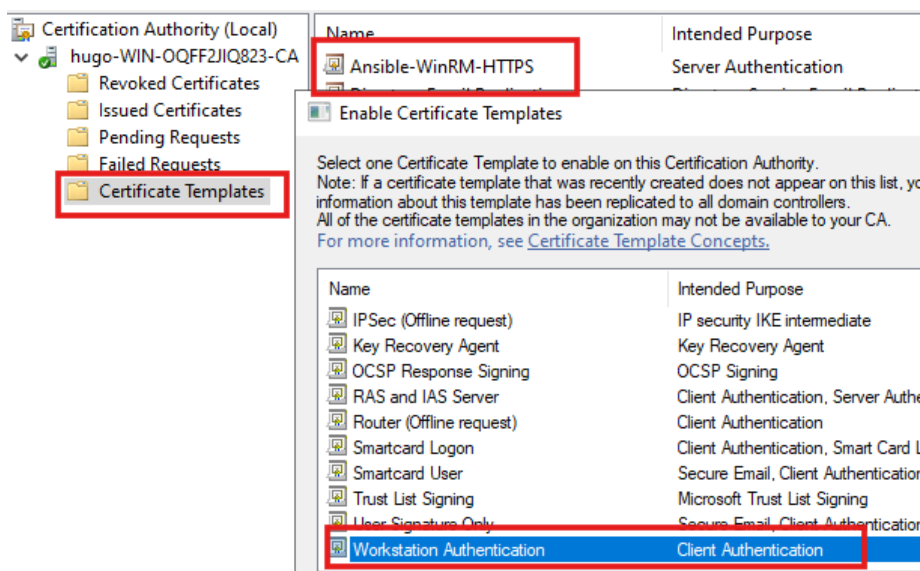
We're not concerned with **UPN** here because this template is for computers, not for users.

What matters to us is the **DNS name**, because WinRM over HTTPS will then require the certificate to match the computer's name.

Now we're going to publish the template in the CA.



We add it to the templates, and now it works:



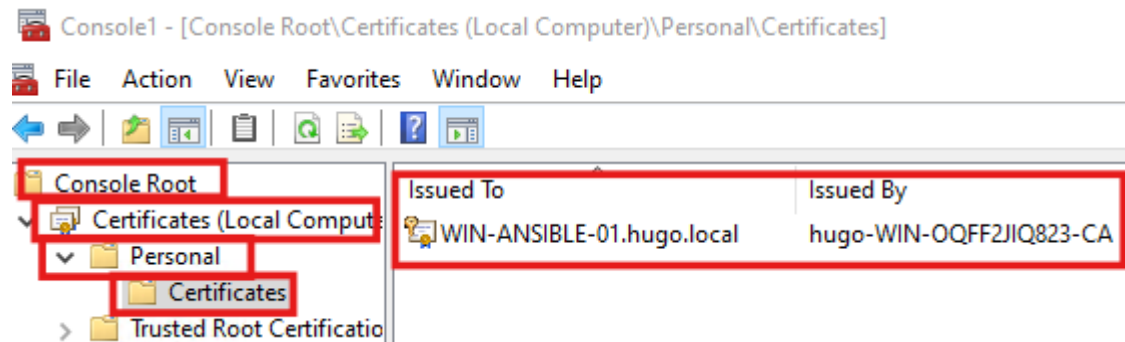
Now let's go back to the **Windows** client to force it to receive the policy; try requesting the certificate using the new template. We'll run the same commands as before:

```
PS C:\Users\Administrator> gpupdate /force
Updating policy...

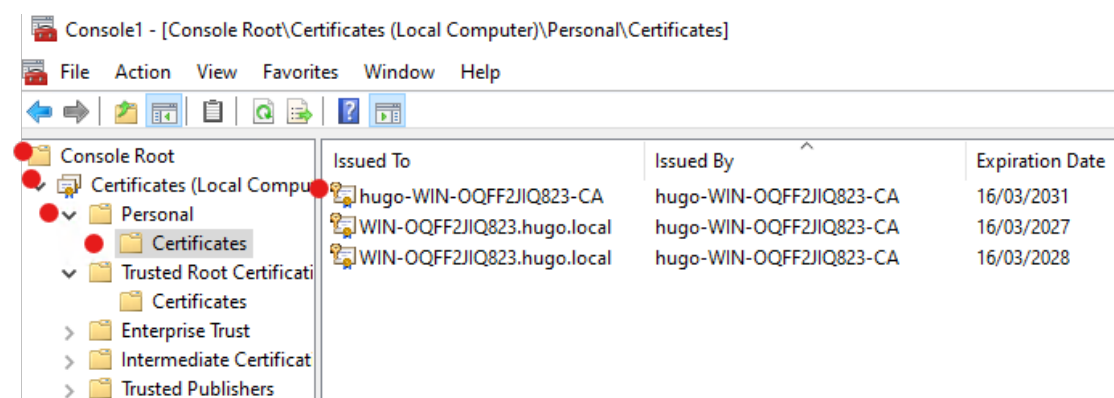
Computer Policy update has completed successfully.
User Policy update has completed successfully.

PS C:\Users\Administrator> certutil -pulse
CertUtil: -pulse command completed successfully.
PS C:\Users\Administrator>
```

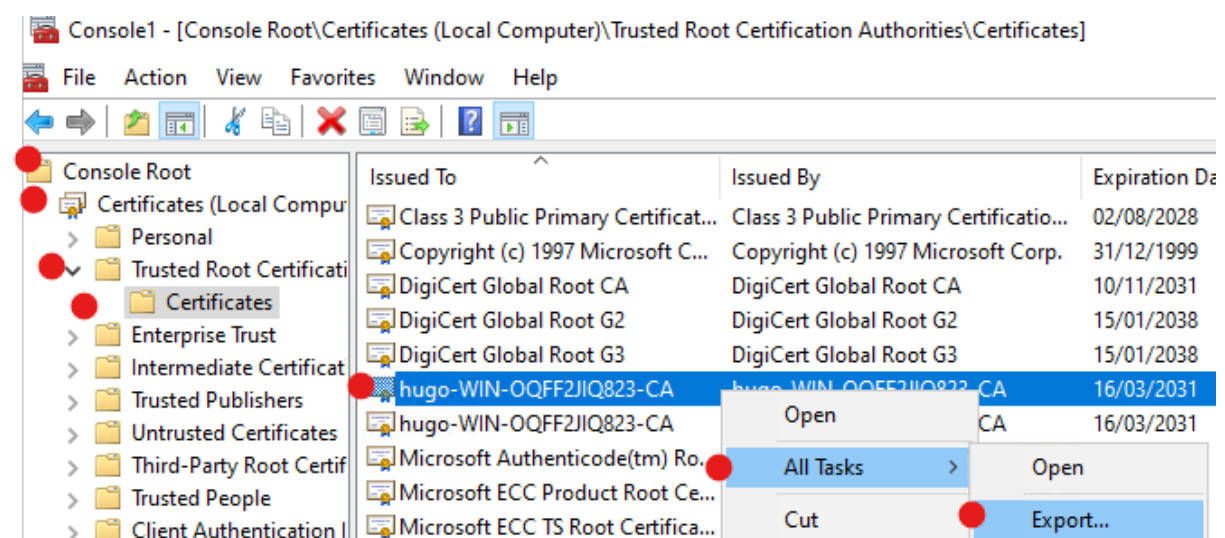
It's the CA certificate, and it's used so that Ubuntu can recognize it as an authentication authority. And as we can see, it's already showing up, so the combination of **GPO, auto-enrollment, the template, and permissions** is working properly.



The client certificate is used to authenticate the client, the connection is encrypted.



What we're going to do is export the certificate and mark it as "trusted" in Ubuntu.



That file is the CA's public certificate, which is exactly what Ubuntu needs to trust the certificates issued by that CA.

Completing the Certificate Export Wizard

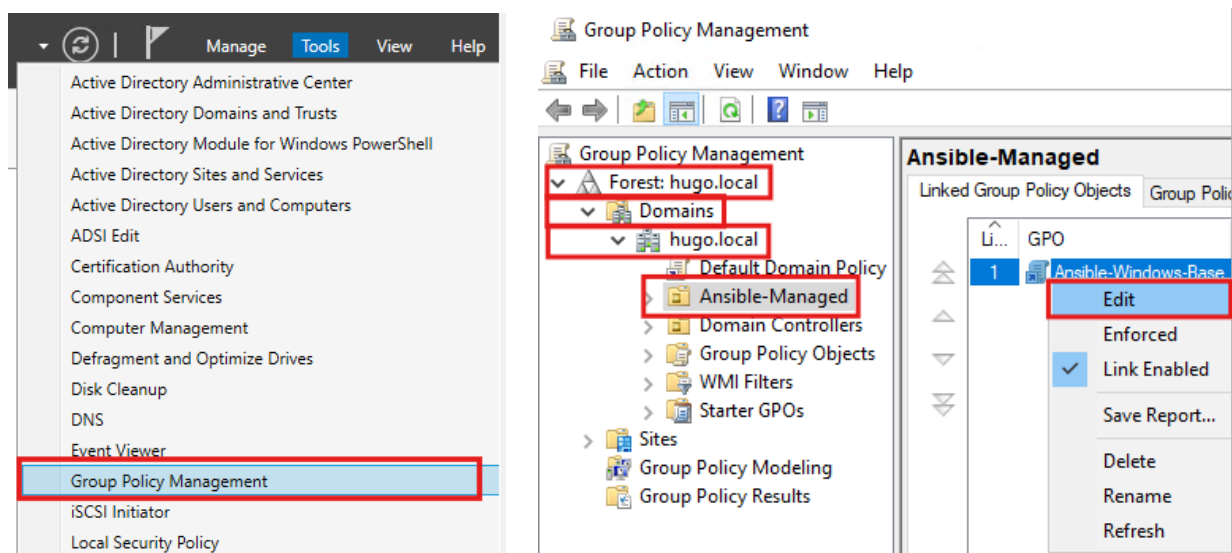
You have successfully completed the Certificate Export wizard.

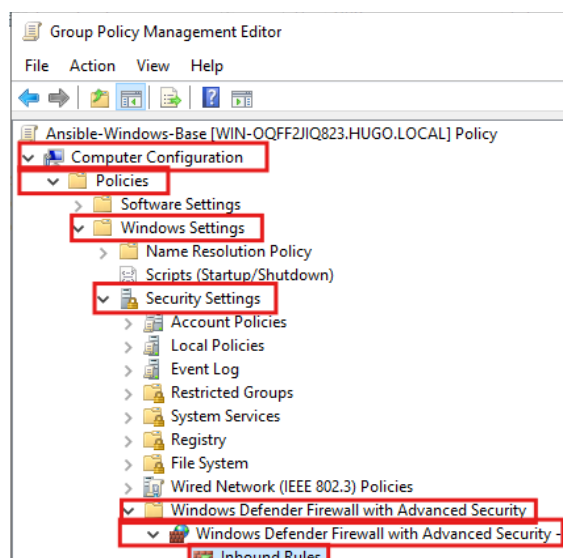
You have specified the following settings:

File Name	C:\Users\Administrator\Desktop\hugo-
Export Keys	No
Include all certificates in the certification path	No
File Format	Base64 Encoded X.509 (*.cer)

6. Firewall Configuration

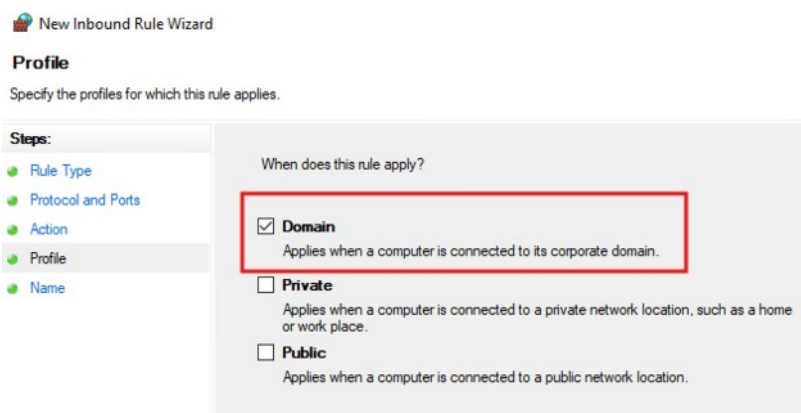
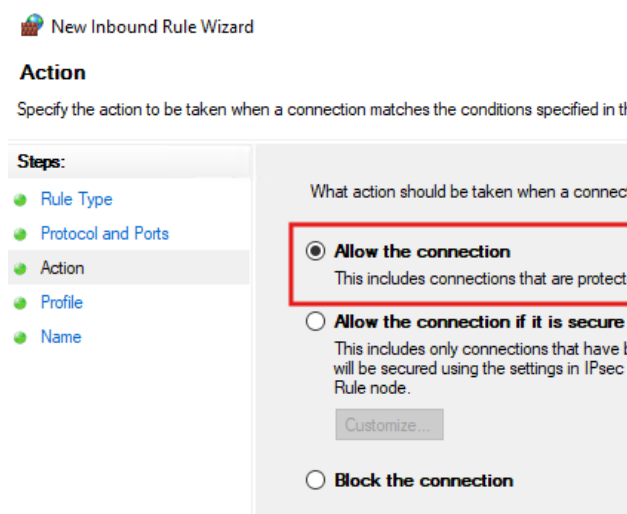
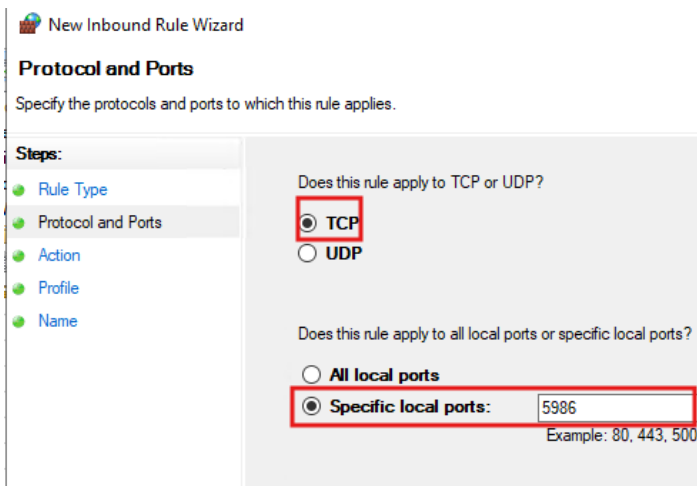
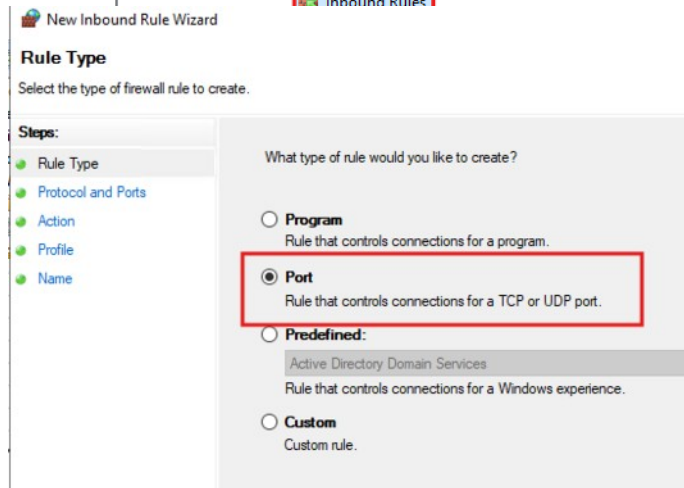
Now we need to create a firewall rule to open port 5986, which will be done using GPO.





GPO → Computer Configuration → Policies → Windows Settings → Security Settings → Windows Defender Firewall with Advanced Security → Inbound Rules

Now we are going to create a new rule to open the port 5986.



New Inbound Rule Wizard

Name

Specify the name and description of this rule.

Steps:

- Rule Type
- Protocol and Ports
- Action
- Profile
- Name

Name:
WinRM-HTTPS-5986

Description (optional):

These are the settings I've applied to the rule.

And after restarting, the rule was created successfully.

Get-NetFirewallRule -PolicyStore RSOP | Where-Object {\$_.DisplayName -like "*WinRM*" -or \$_.DisplayName -like "*5986*"} | Format-List DisplayName, Enabled, Direction, Action, PolicyStoreSource, PolicyStoreSourceType

```
PS C:\Users\Administrator> Get-NetFirewallRule -PolicyStore RSOP | Where-Object {$_.DisplayName -like "*WinRM*" -or $_.DisplayName -like "*5986*"} | Format-List DisplayName, Enabled, Direction, Action, PolicyStoreSource, PolicyStoreSourceType

DisplayName      : WinRM-HTTPS-5986
Enabled          : True
Direction        : Inbound
Action           : Allow
PolicyStoreSource : 
PolicyStoreSourceType : GroupPolicy
PS C:\Users\Administrator>
```

7. Trusting the CA in Ubuntu

Now I'm exporting it to the Ubuntu machine:

scp C:\Users\Administrator\Desktop\hugo-ca.cer hugo@10.34.1.115:/home/hugo/

```
PS C:\Users\Administrator> scp C:\Users\Administrator\Desktop\hugo-ca.cer hugo@10.34.1.115:/home/hugo/
The authenticity of host '10.34.1.115 (10.34.1.115)' can't be established.
ECDSA key fingerprint is SHA256:LStIvfpGlskNBWYnJEr8RxianpRbHEtQb7ud2NVw02Q.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added '10.34.1.115' (ECDSA) to the list of known hosts.
hugo@10.34.1.115's password:
hugo-ca.cer                                     100% 1286    1.3KB/s   00:00
PS C:\Users\Administrator>
```

We imported 2 certificates (from DEV and PROD) into Ubuntu so the system would trust them.

```
hugo@hugo-server:~$ ls
ansible-windows-lab  hosts.ini  hugo-ca.cer
```

The certificate files must use the .cer extension. After copying them to the system trust store, the certificate database must be updated. This makes Ubuntu add that certificate to its trusted CA store.

```
sudo cp ~/hugo-ca.cer /usr/local/share/ca-certificates/hugo-ca-prod.crt
sudo cp ~/hugo-ca.cer /usr/local/share/ca-certificates/hugo-ca-dev.crt
```

```
sudo update-ca-certificates
```

The file `/etc/ssl/certs/ca-certificates.crt` is not a single certificate. It is a **bundle of trusted CA certificates** generated by the system, and it can contain multiple certificates in the same file. By pointing `REQUESTS_CA_BUNDLE` to this file, Python and Ansible can validate server certificates against all trusted certification authorities installed in Ubuntu.

```
nano ~/.bashrc
export REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt
```

We do this so Python and Ansible use Ubuntu's trusted CA bundle when validating HTTPS certificates. By adding that variable to `~/.bashrc` the variable is loaded automatically in each session.

8. Ansible Script

This playbook configures WinRM on Windows to use **HTTPS only** with a valid machine certificate, applies the defined network filters, removes the **HTTP** listener, restarts the service, and finally verifies that the configuration is correct. It also saves a process log in **C:\Windows\Temp\ansible-winrm-log.txt**.

SCRIPT

(No es público por temas relacionados con la empresa, si lo necesitas contáctame)

The script is executed manually once for each machine group, which is sufficient for this deployment model.

```
ansible@ansible-controller:~/ansible$ ansible-playbook -i inventories/dev/hosts playbooks/winrm-https-setup.yml --ask-vault-pass
Vault password:
```

ansible windows -i inventory.ini -m ansible.windows.win_ping

```
(.venv) hugo@hugo-server:~/ansible-windows-lab$ ansible windows -i inventory.ini -m ansible.windows.win_ping
winserver | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

9. Maintenance

In this project, maintenance must be carried out every **two years**, since that is the validity period of the certificates used for WinRM over HTTPS.

Each HTTPS listener is linked to the certificate that was assigned to the machine at that time. When the certificate expires and a new one is issued, the existing listener may still be associated with the old certificate. For this reason, the HTTPS listener must be removed and the configuration script must be executed again so that WinRM is configured with the new valid certificate.

This maintenance ensures that remote management continues to work correctly and securely over HTTPS.

Command to delete the HTTPS listener:

winrm delete winrm/config/Listener?Address=&Transport=HTTPS*

After deleting the listener, the configuration script must be run again.